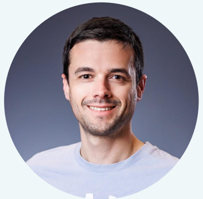
Kubernetes at the Edge
See it in action

Cloud Native Denmark | 08/10/2025

Antonia von den Driesch
Platform Engineer



Xavier Avrillier
Solutions Architect

What is Edge Computing?

Why Kubernetes at the Edge?

The **KubeEdge** project

Product architecture and components

DEMO

Reality check

Q&A

EDGE COMPUTING

Edge Computing

“Edge computing is part of a distributed computing topology where information processing is located close to the edge, where things and people produce or consume that information.

Gartner glossary

What is Edge Computing

What is Edge Computing

Why Edge Computing

Latency

Bandwidth

Autonomy

Privacy

Why Edge Computing

Latency

Super quick decisions (self driving cars)

Bandwidth

Too much raw data to stream all to cloud

Autonomy

Must keep working when network is down

Privacy

Sensitive data must stay local

Industrial IoT
predictive maintenance, safety, QA...

Retail
footfall counting, queue management...

Smart Cities
traffic optimization, surveillance cameras...

Telco/5G
base station intelligence, reducing data sent over backhaul networks...

Healthcare
in-ambulance data processing, patient data confidentiality...

Space
satellite in-orbit pre-processing

KUBERNETES AT THE EDGE

WHY ?

Challenges at the Edge

Scale

Connectivity

Updates

Challenges at the Edge

Scale

Thousands of nodes and devices across multiple locations

Connectivity

Remote, resource-constrained and sometimes offline

Updates

Needs consistent updates and patches under unreliable conditions

Kubernetes at the Edge

- Declarative
- Portable
- Automated
- Familiar
- But

Kubernetes at the Edge

	Declarative	*Reconcile a desired state*
	Portable	
	Automated	
	Familiar	
	But	

Kubernetes at the Edge

Declarative	*Reconcile a desired state*
Portable	*Same apps can run in cloud, datacenter, or edge*
Automated	
Familiar	
But	

Kubernetes at the Edge

Declarative	*Reconcile a desired state*
Portable	*Same apps can run in cloud, datacenter, or edge*
Automated	*Reducing manual ops*
Familiar	
But	

Kubernetes at the Edge

	Declarative	*Reconcile a desired state*
	Portable	*Same apps can run in cloud, datacenter, or edge*
	Automated	*Reducing manual ops*
	Familiar	*Same operational model as in the cloud*
	But	

Kubernetes at the Edge

	Declarative	*Reconcile a desired state*
	Portable	*Same apps can run in cloud, datacenter, or edge*
	Automated	*Reducing manual ops*
	Familiar	*Same operational model as in the cloud*
	But	*Assumption of stable networking*

KUBEEDGE Project

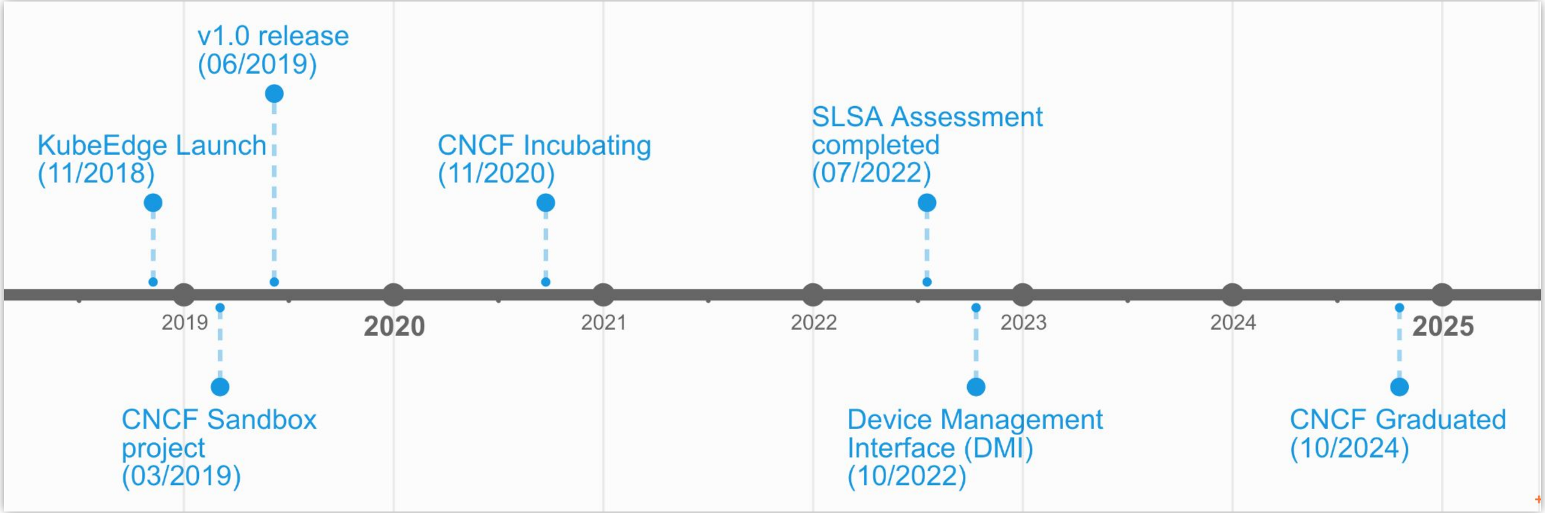
KubeEdge

7.2k stars

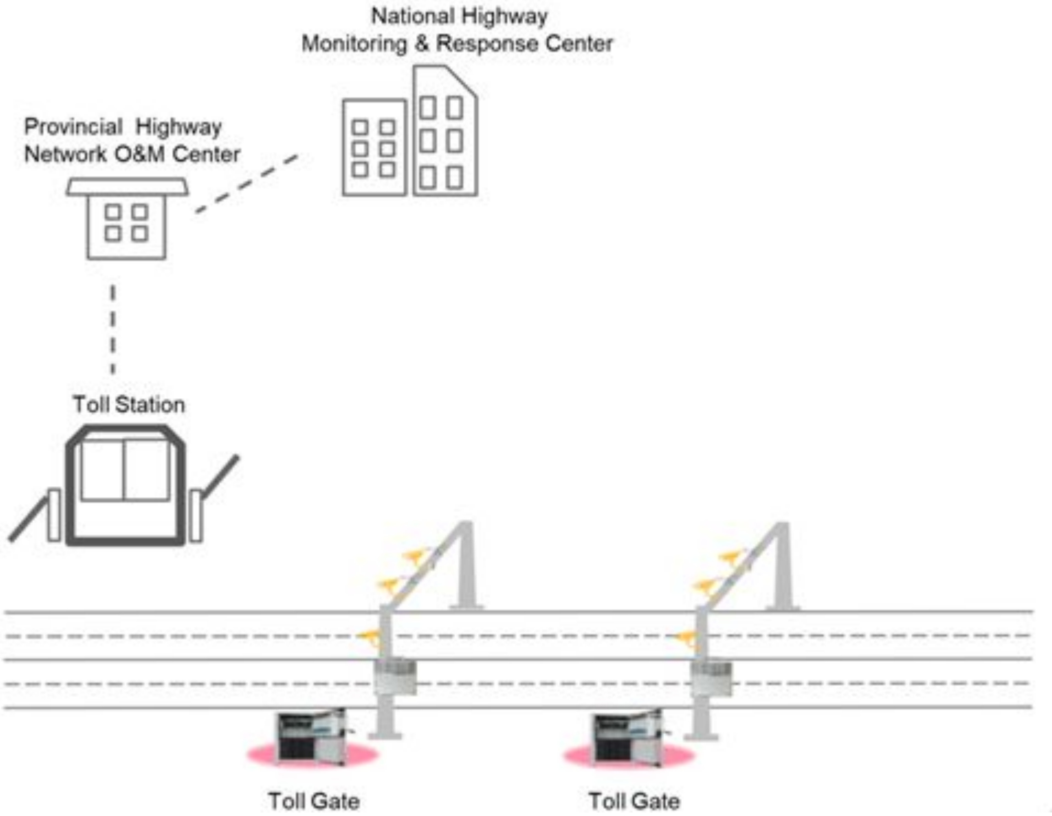
181 watching

1.8k forks

Latest release as of the time of this talk: **1.21.0**

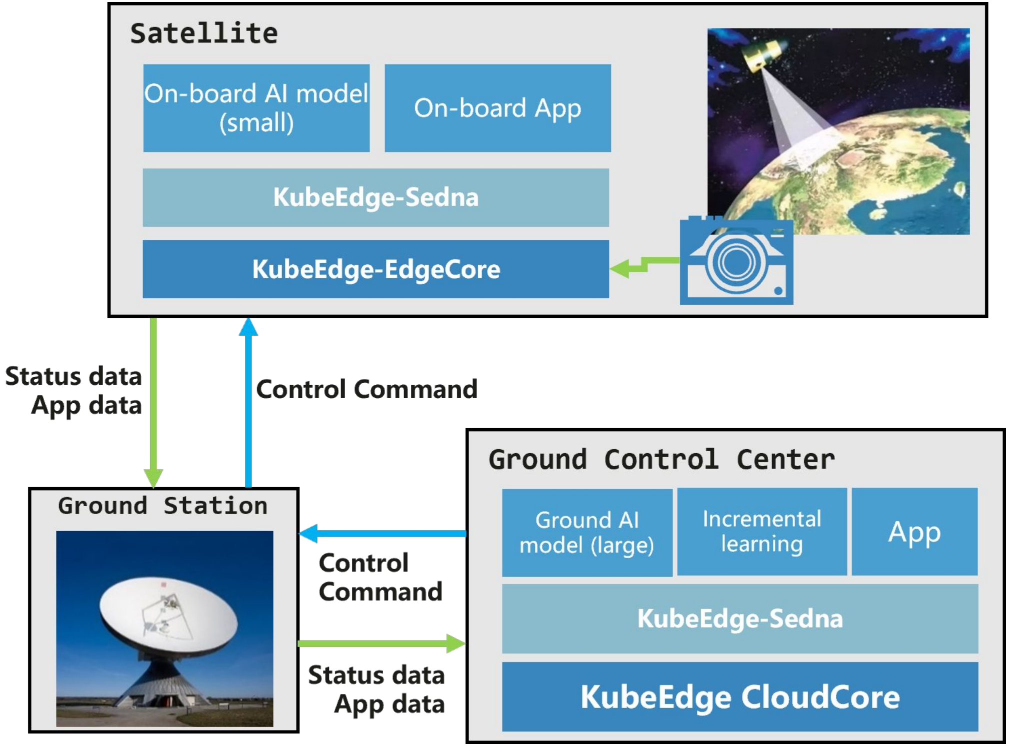


China highway electronic toll collection



Over 100,000 gantries, controllers, and edge devices deployed on highways to achieve one unified network and remove segregated, provincial toll stations.

First Cloud Native edge computing satellite

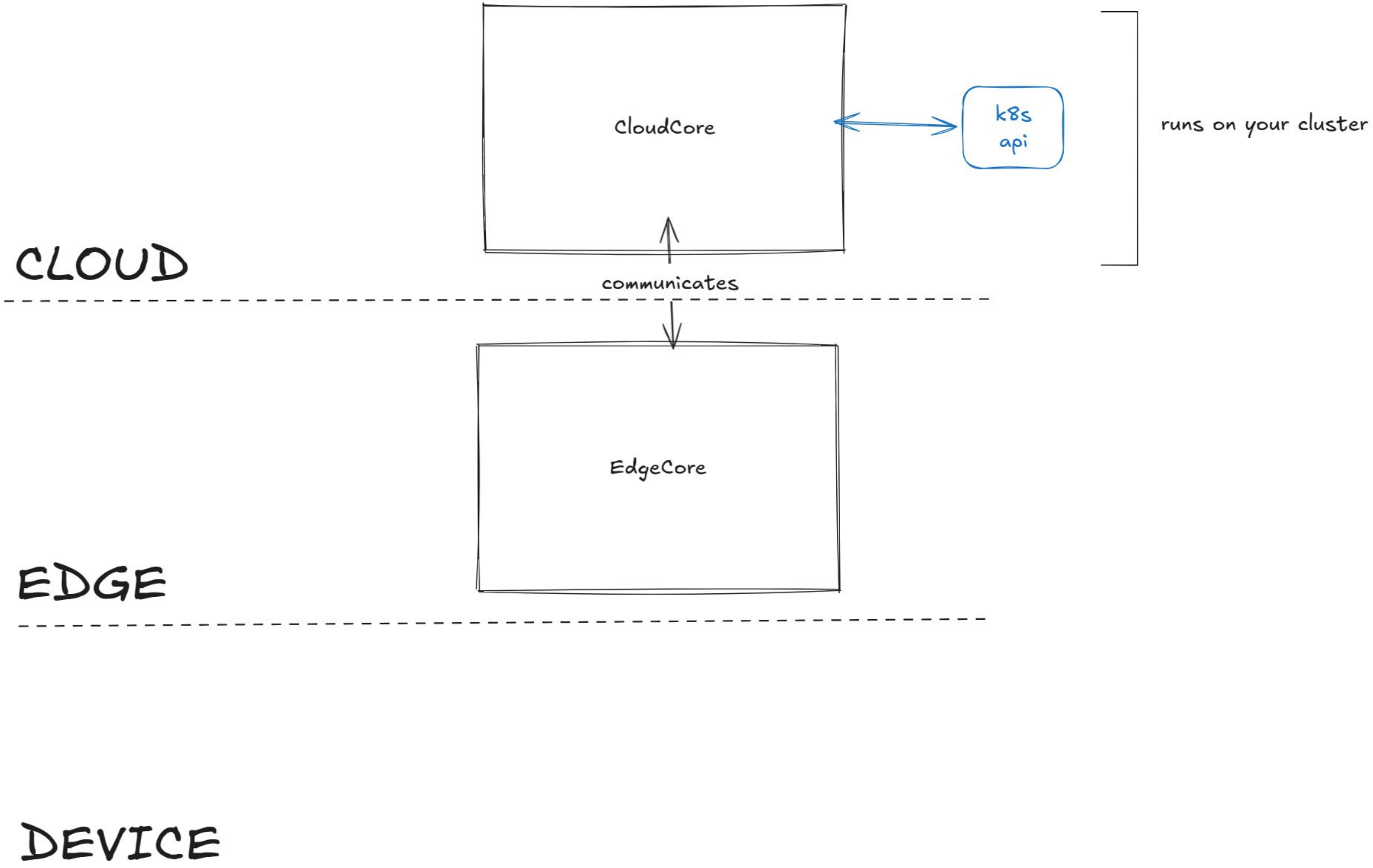


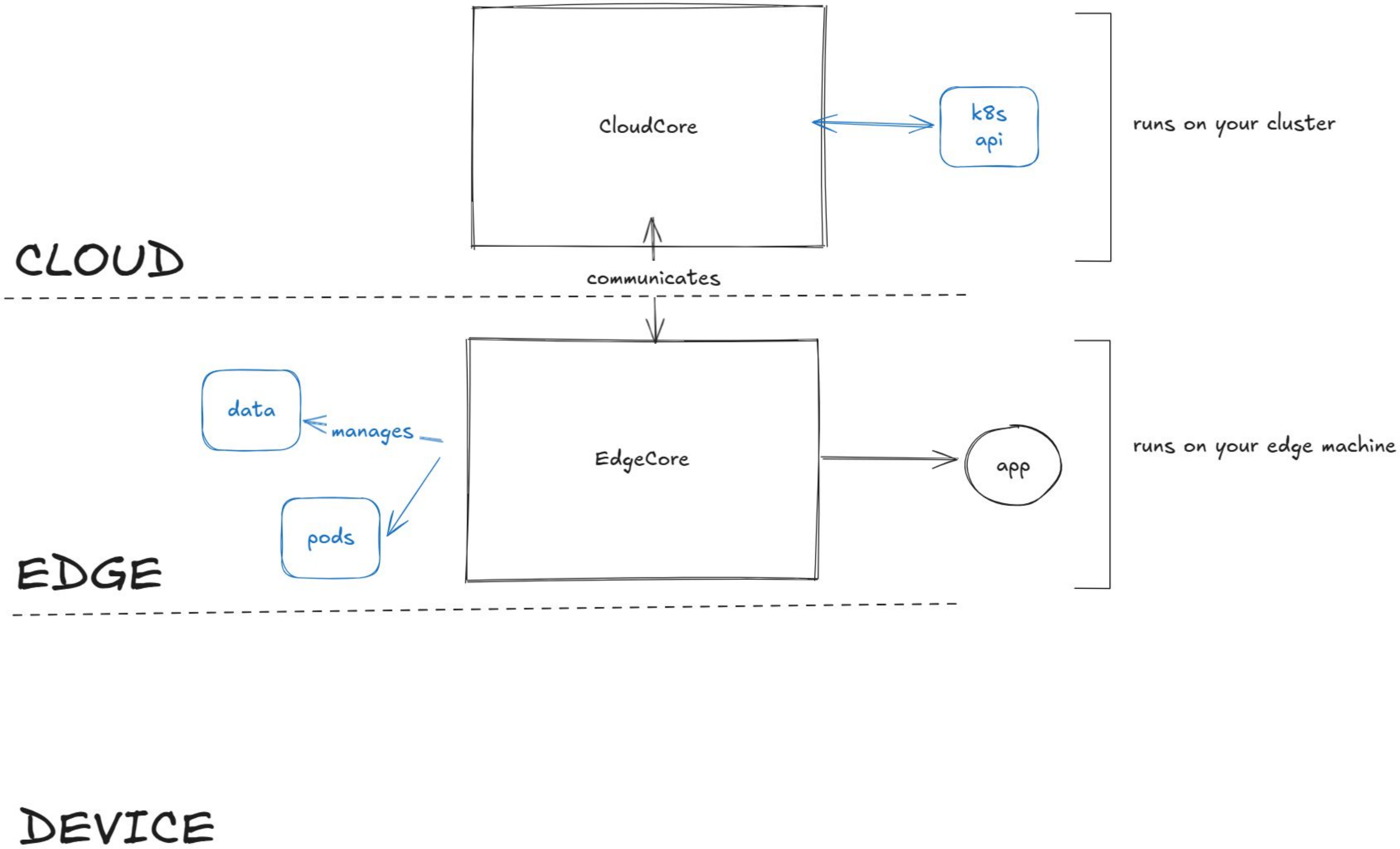
Need for in-orbit compute processing due to increase in data generation vs downlink bandwidth.

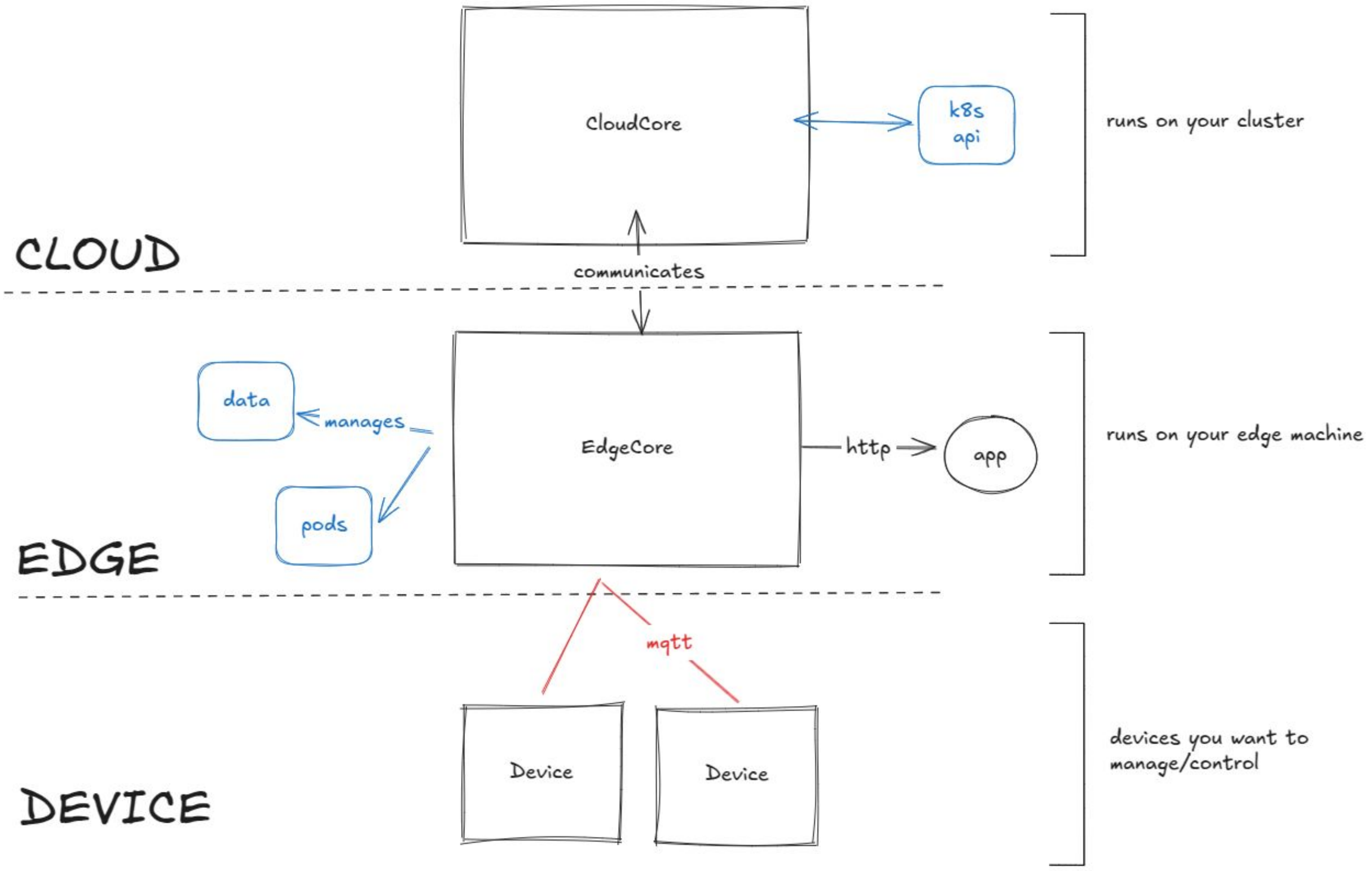
More case studies at

kubeedge.io/case-studies

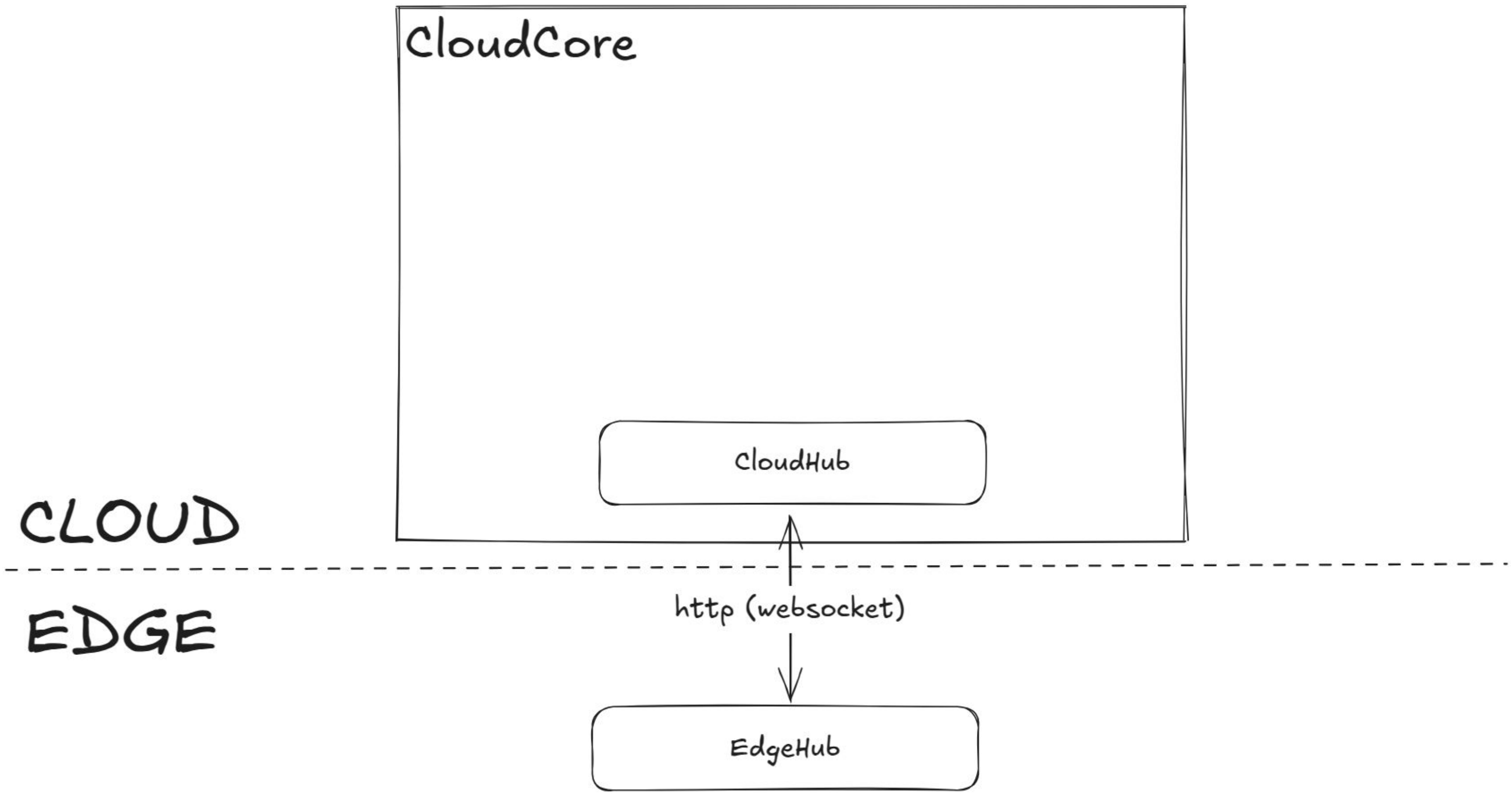
ARCHITECTURE

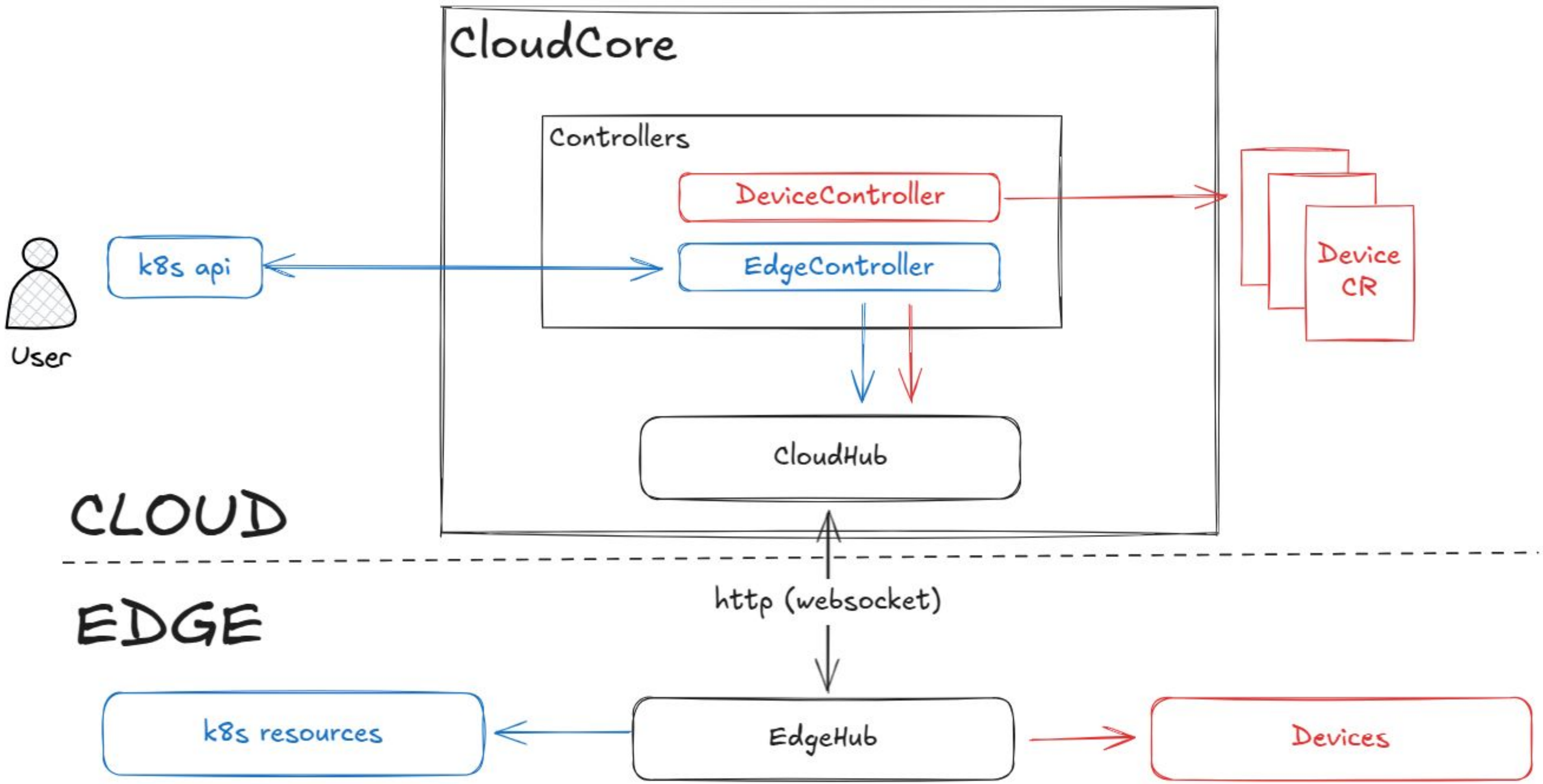

CloudCore
k8s
api
runs on your cluster
CLOUD
communicates
EdgeCore
EDGE
DEVICE


CLOUD
CloudCore
k8s api
runs on your cluster
communicates
EDGE
data
manages
pods
EdgeCore
app
runs on your edge machine
DEVICE

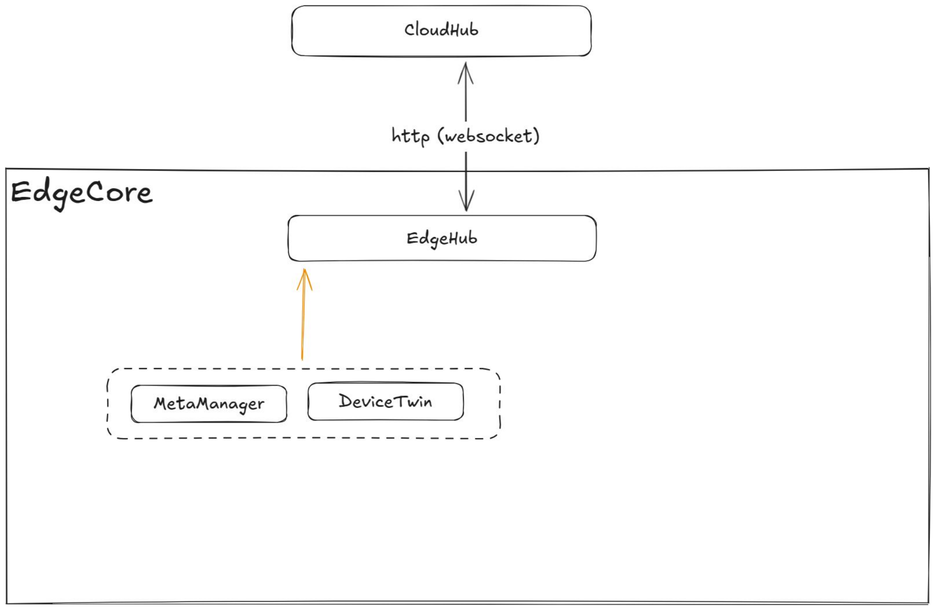

CLOUD
CloudCore
k8s
api
runs on your cluster
communicates
EDGE
EdgeCore
data
manages
pods
http
app
runs on your edge machine
mqtt
DEVICE
Device
Device
devices you want to
manage/control

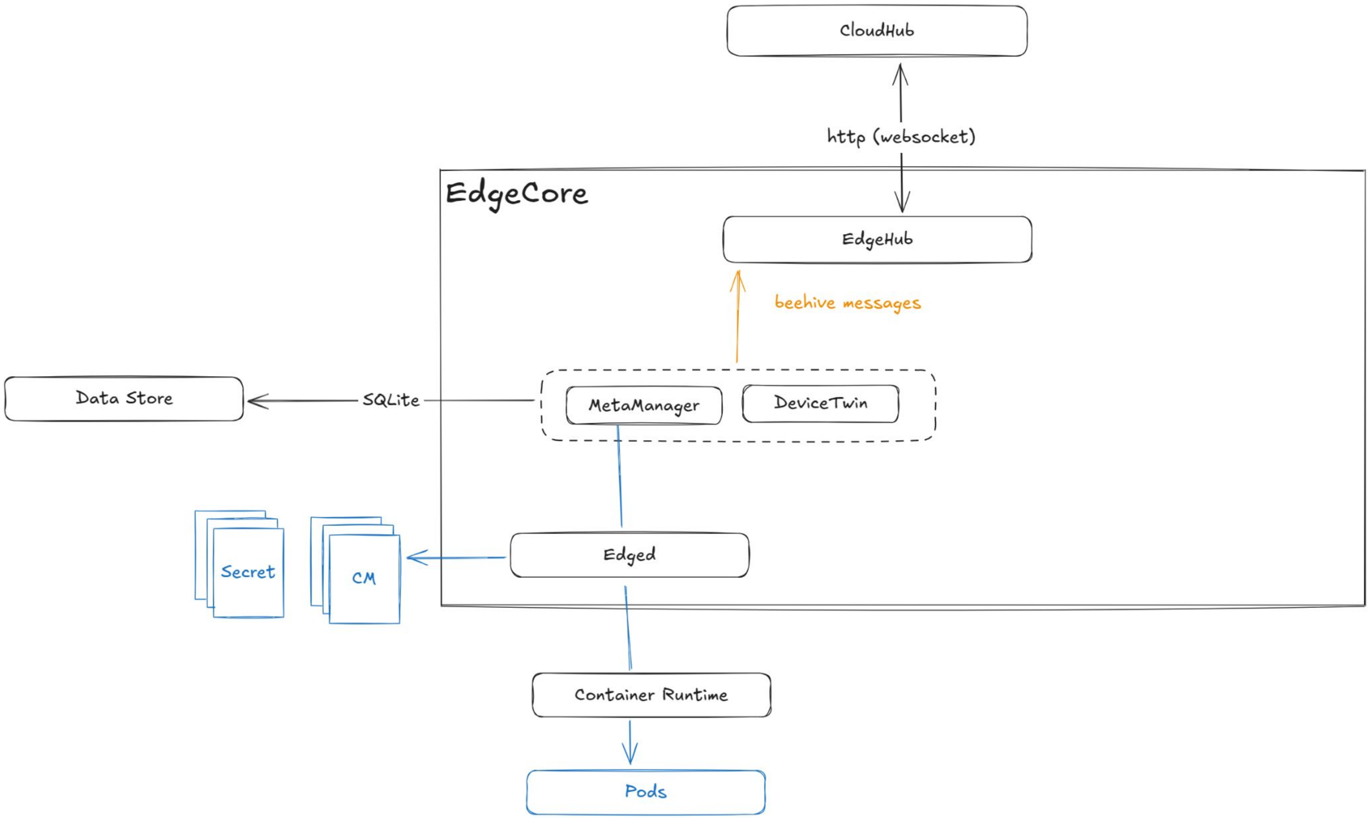
CLOUD CORE

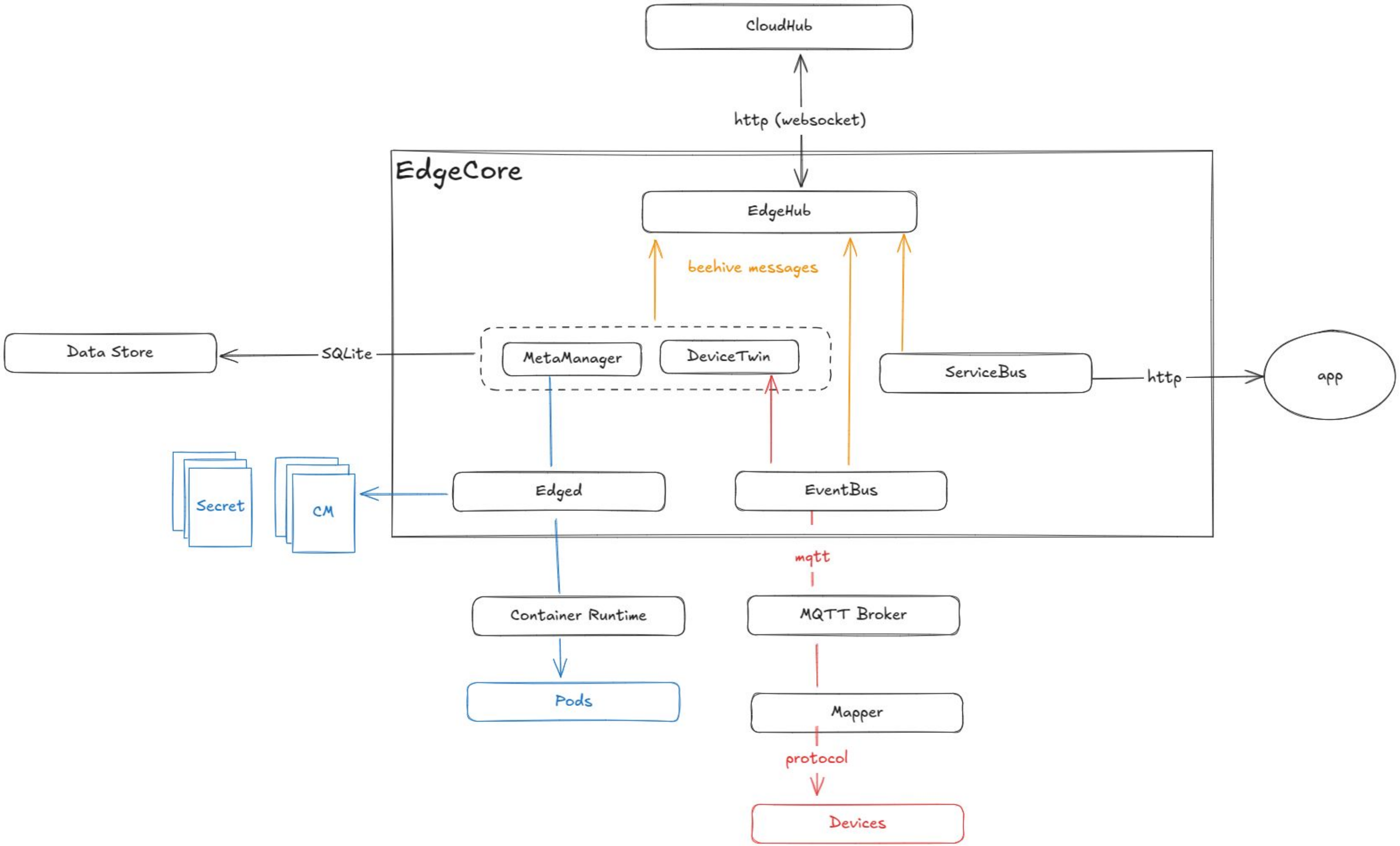

CloudCore
CloudHub
CLOUD
http (websocket)
EDGE
EdgeHub


CloudCore
Controllers
DeviceController
EdgeController
Device CR
User
k8s api
CloudHub
CLOUD
EDGE
http (websocket)
k8s resources
EdgeHub
Devices

EDGE CORE


CloudHub
http (websocket)
EdgeCore
EdgeHub
MetaManager
DeviceTwin


CloudHub
http (websocket)
EdgeCore
EdgeHub
beehive messages
Data Store
SQLite
MetaManager
DeviceTwin
Secret
CM
Edged
Container Runtime
Pods

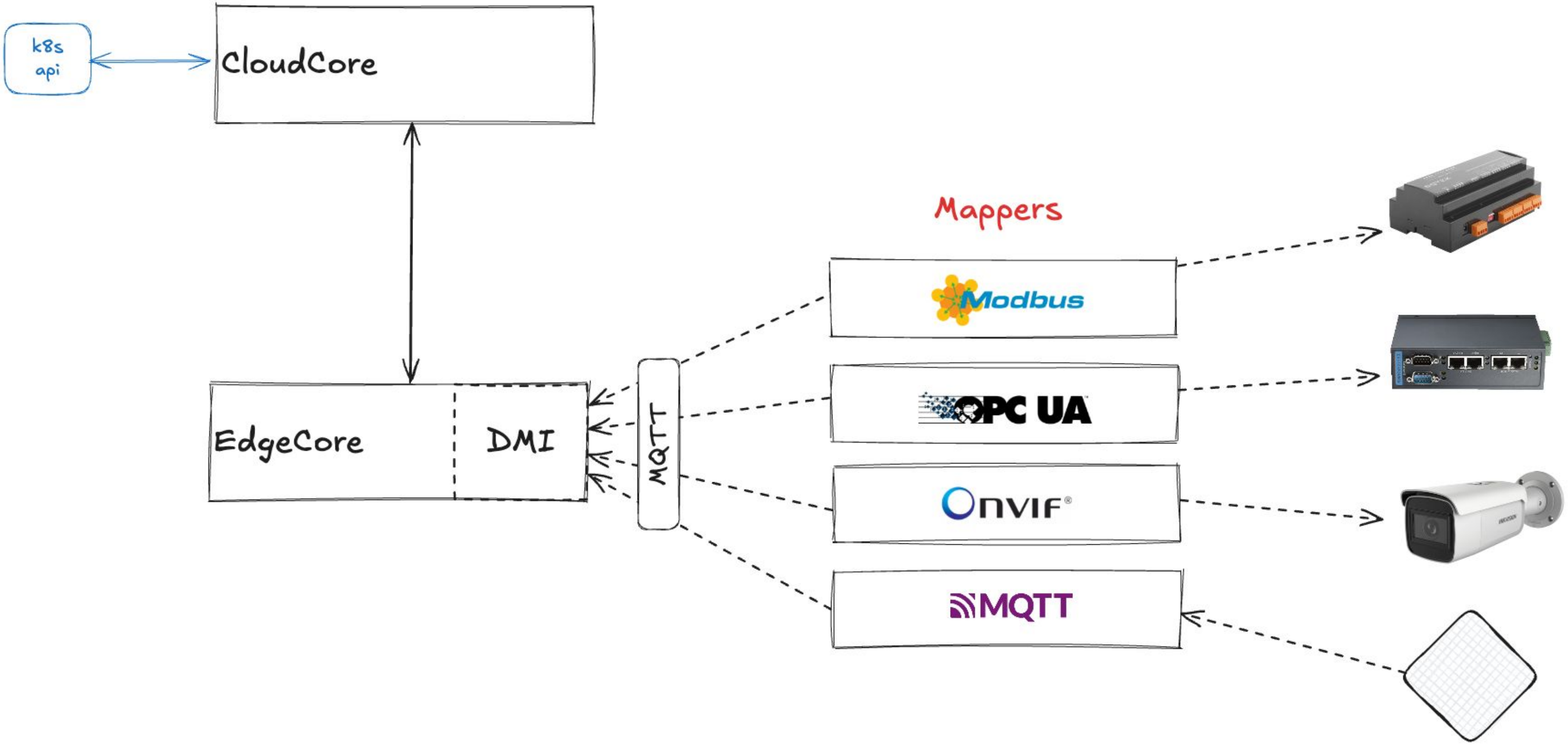


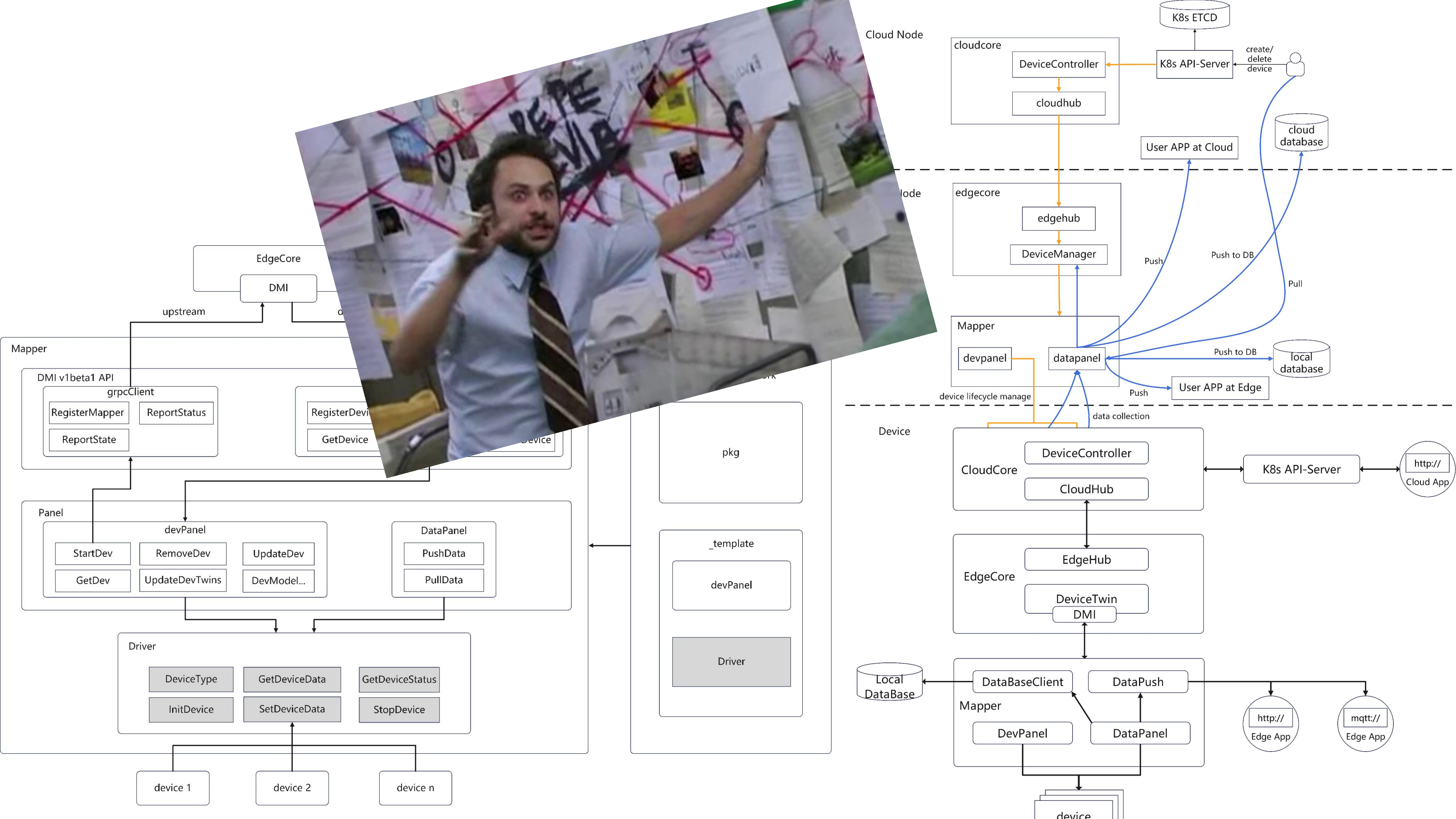
CloudHub
http (websocket)
EdgeCore
EdgeHub
beehive messages
Data Store
SQLite
MetaManager
DeviceTwin
ServiceBus
http
app
Secret
CM
Edged
EventBus
mqtt
Container Runtime
MQTT Broker
Pods
Mapper
protocol
Devices

DEVICES (IIOT)

It's all about protocols

Mappers are translators between the physical world (device protocols) and the KubeEdge logical world


k8s
api
CloudCore
Mappers
Modbus
OPC UA
ONVIF
MQTT
EdgeCore
DMI
MQTT


EdgeCore
DMI
upstream
Mapper
DMI v1beta1 API
grpcClient
RegisterMapper
ReportStatus
ReportState
RegisterDevi
GetDevice
Device
Panel
devPanel
StartDev
RemoveDev
UpdateDev
GetDev
UpdateDevTwins
DevModel...
DataPanel
PushData
PullData
Driver
DeviceType
GetDeviceData
GetDeviceStatus
InitDevice
SetDeviceData
StopDevice
device 1
device 2
device n
pkg
_template
devPanel
Driver
Cloud Node
cloudcore
DeviceController
cloudhub
K8s ETCD
K8s API-Server
create/
delete
device
User APP at Cloud
cloud
database
edgecore
edgehub
DeviceManager
Push
Push to DB
Pull
Mapper
devpanel
datapanel
Push to DB
local
database
Push
User APP at Edge
device lifecycle manage
data collection
Device
CloudCore
DeviceController
CloudHub
K8s API-Server
http://
Cloud App
EdgeCore
EdgeHub
DeviceTwin
DMI
Local
DataBase
DataBaseClient
DataPush
Mapper
DevPanel
DataPanel
http://
Edge App
mqtt://
Edge App
device

Edge Device CRDs (Custom Resources Definitions)

Device Models

Physical model properties and parameters based on capabilities

Device Instances

A device instance represents an actual device object.
Specifies each property, including its name, type, access method…

Device Model example

Advantech ADAM-6017

8-channel (connect sensors)

Modbus

Ethernet

KubeEdge Device Model CR

Giant Swarm

```yaml
apiVersion: devices.kubeedge.io/v1beta1
kind: DeviceModel
metadata:
  name: adam6017-model
spec:
  properties:
    - name: channel0
      description: "Analog input channel 0"
      type: INT
      accessMode: ReadOnly
      unit: mV
    - name: channel1
      description: "Analog input channel 1"
      type: INT
      accessMode: ReadOnly
      unit: mV
    - name: ...
  protocol: modbus
```

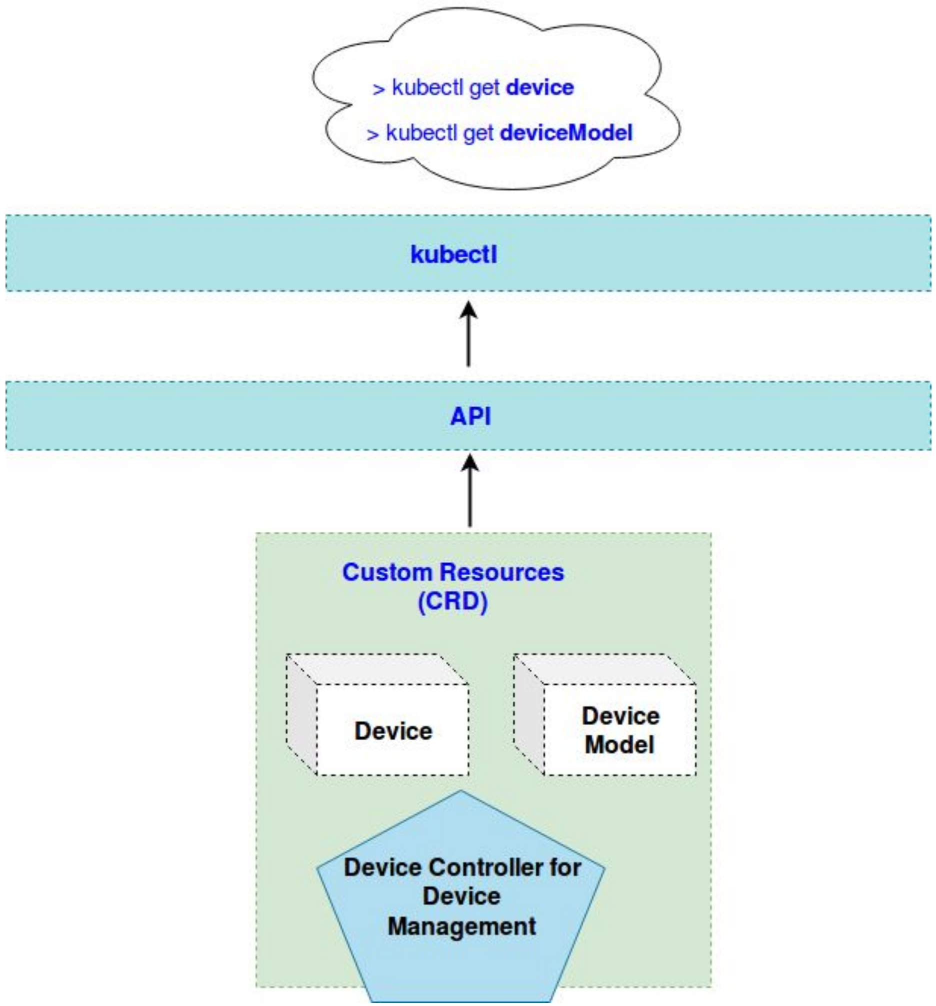
KubeEdge Device Instance CR

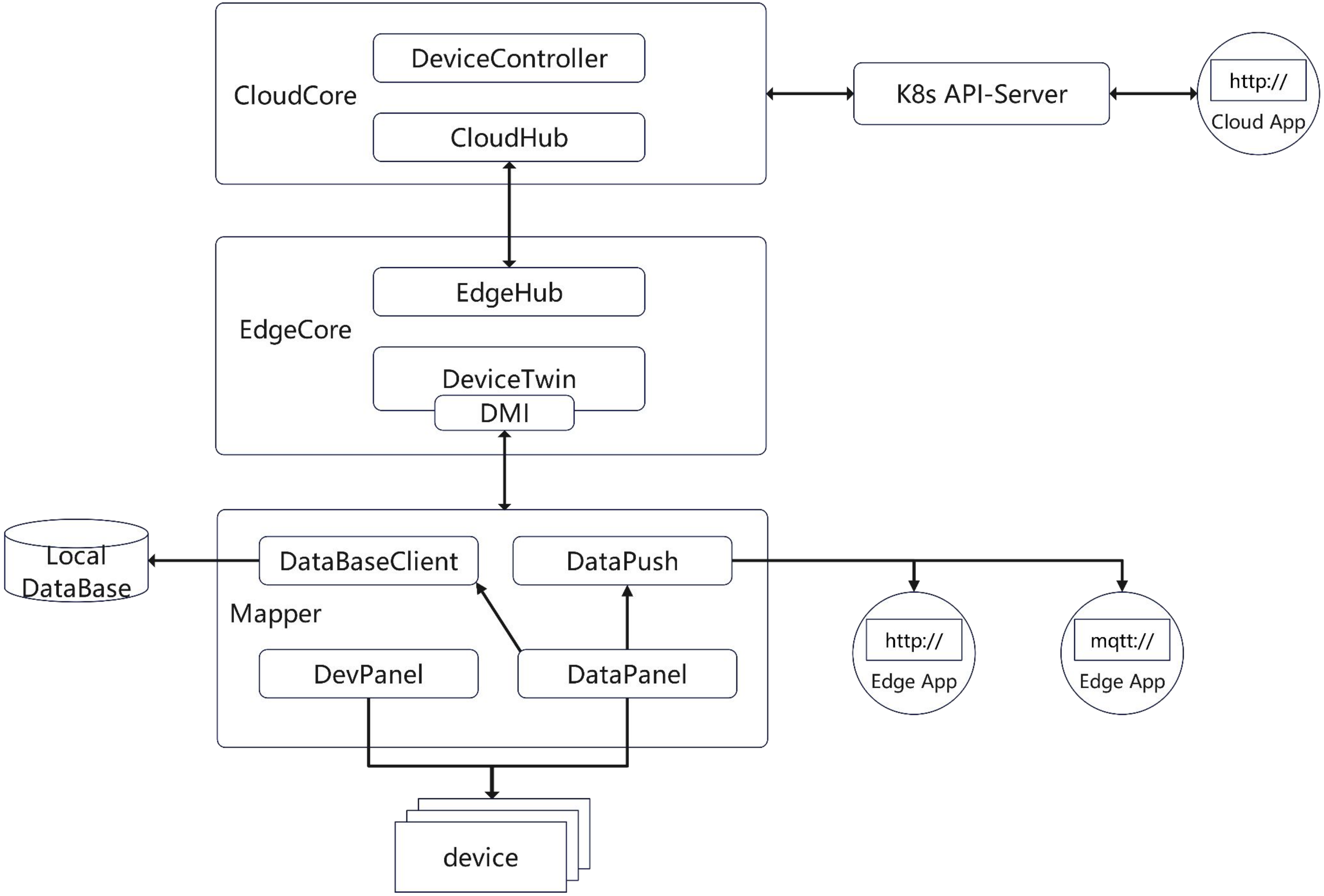
Giant Swarm

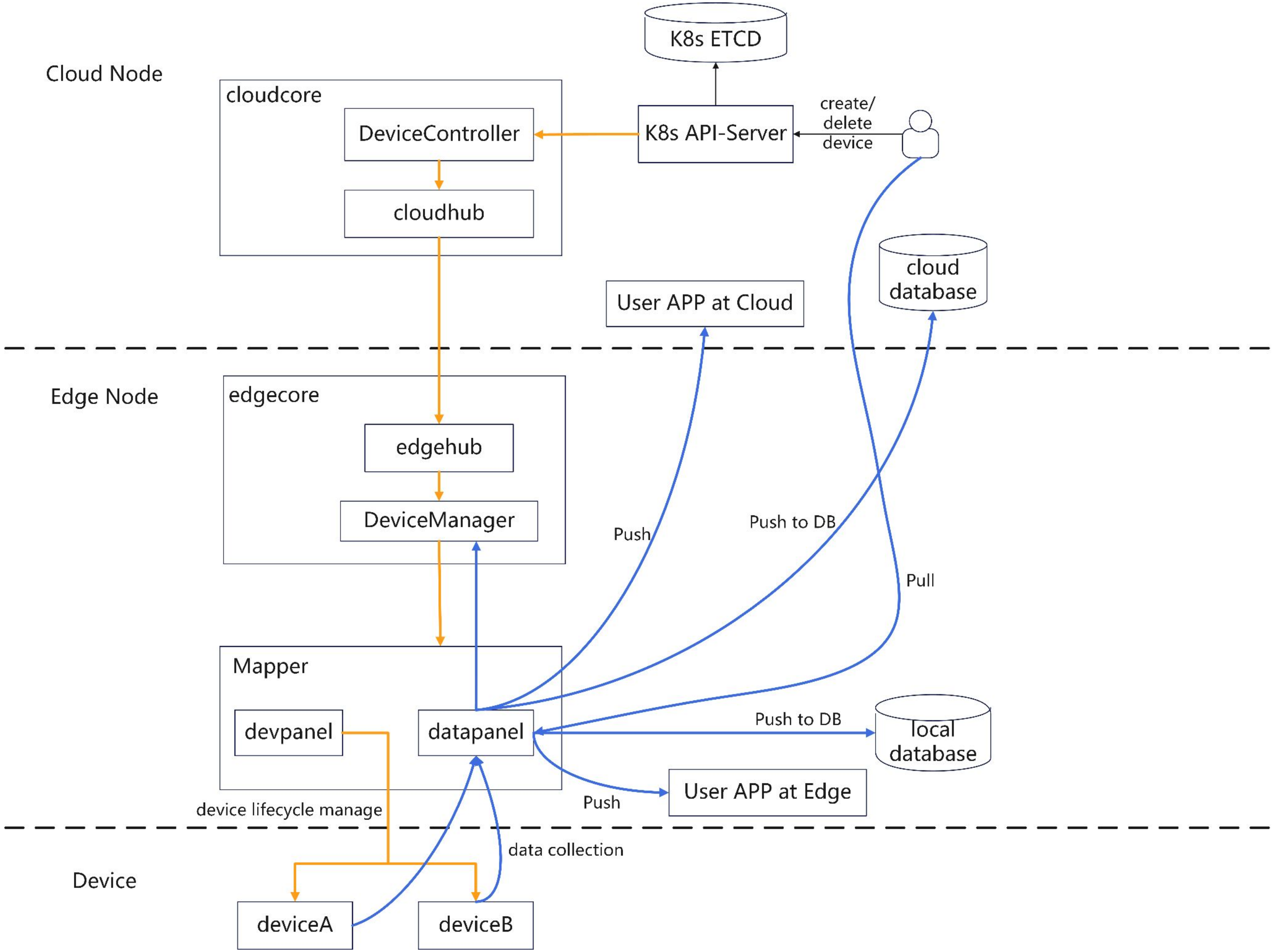
```yaml
apiVersion: devices.kubeedge.io/v1beta1
kind: Device
metadata:
  name: factory1-line1-adam6017
spec:
 deviceModelRef:
   name: adam6017-model
 nodeName: worker-node1
 protocol:
   protocolName: modbus
   configData:
     ip: 172.17.0.3
     port: 1502
```

```yaml
properties:
  - name: channel0
    visitor:
      protocolName: modbus
      register: 40001
      quantity: 1
      functionCode: 3
    pushMethod: []
  - name: channel1
    visitor:
      protocolName: modbus
      register: 40002
      quantity: 1
      functionCode: 3
    pushMethod: []
```

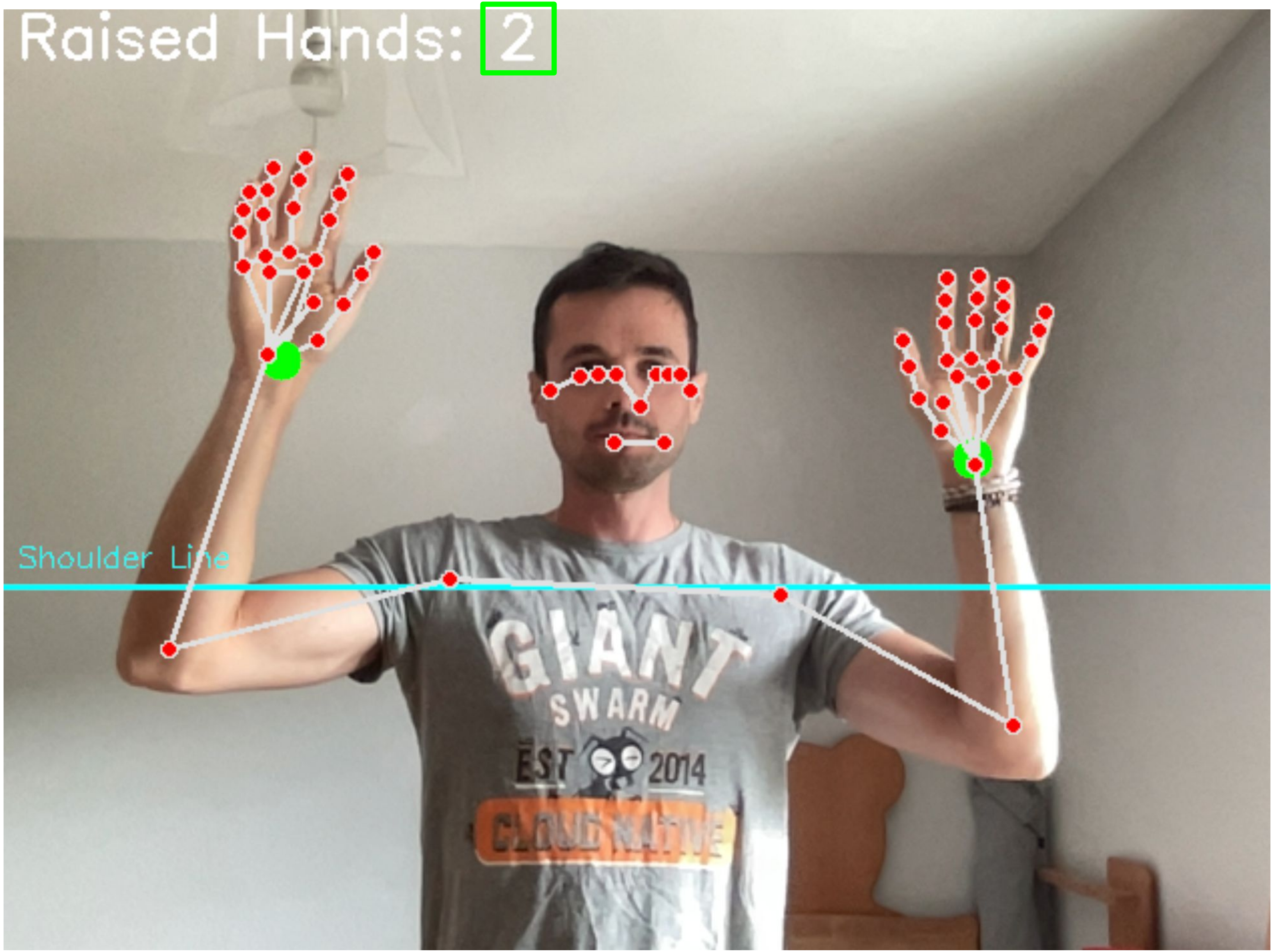
Modbus registers list collected from ADAM-6017 user guide


> kubectl get device
> kubectl get deviceModel
kubectl
API
Custom Resources (CRD)
Device
Device Model
Device Controller for Device Management


CloudCore
DeviceController
CloudHub
K8s API-Server
http://
Cloud App
EdgeCore
EdgeHub
DeviceTwin
DMI
Local DataBase
Mapper
DataBaseClient
DataPush
DevPanel
DataPanel
http://
Edge App
mqtt://
Edge App
device


K8s ETCD
Cloud Node
cloudcore
DeviceController
K8s API-Server
create/
delete
device
cloudhub
cloud
database
User APP at Cloud
Edge Node
edgecore
edgehub
DeviceManager
Push
Push to DB
Pull
Mapper
devpanel
datapanel
Push to DB
local
database
User APP at Edge
Push
device lifecycle manage
data collection
Device
deviceA
deviceB

DEMO TIME


Raised Hands: 2
Shoulder Line
GIANT
SWARM
EST 2014
CLOUD NATIVE

SKIPPED SLIDE - JUST INFORMATIONAL

Open grafana dashboard

`kubectl port-forward -n monitoring svc/grafana 3000:80`

`Go to` http://127.0.0.1:3000/

Do the hands thing and show counter change

Maybe invite Marcus, Joe and Dom to join or point the camera to the first row ? (with caveat about reliability I mention in comments)

SKIPPED SLIDE - JUST INFORMATIONAL

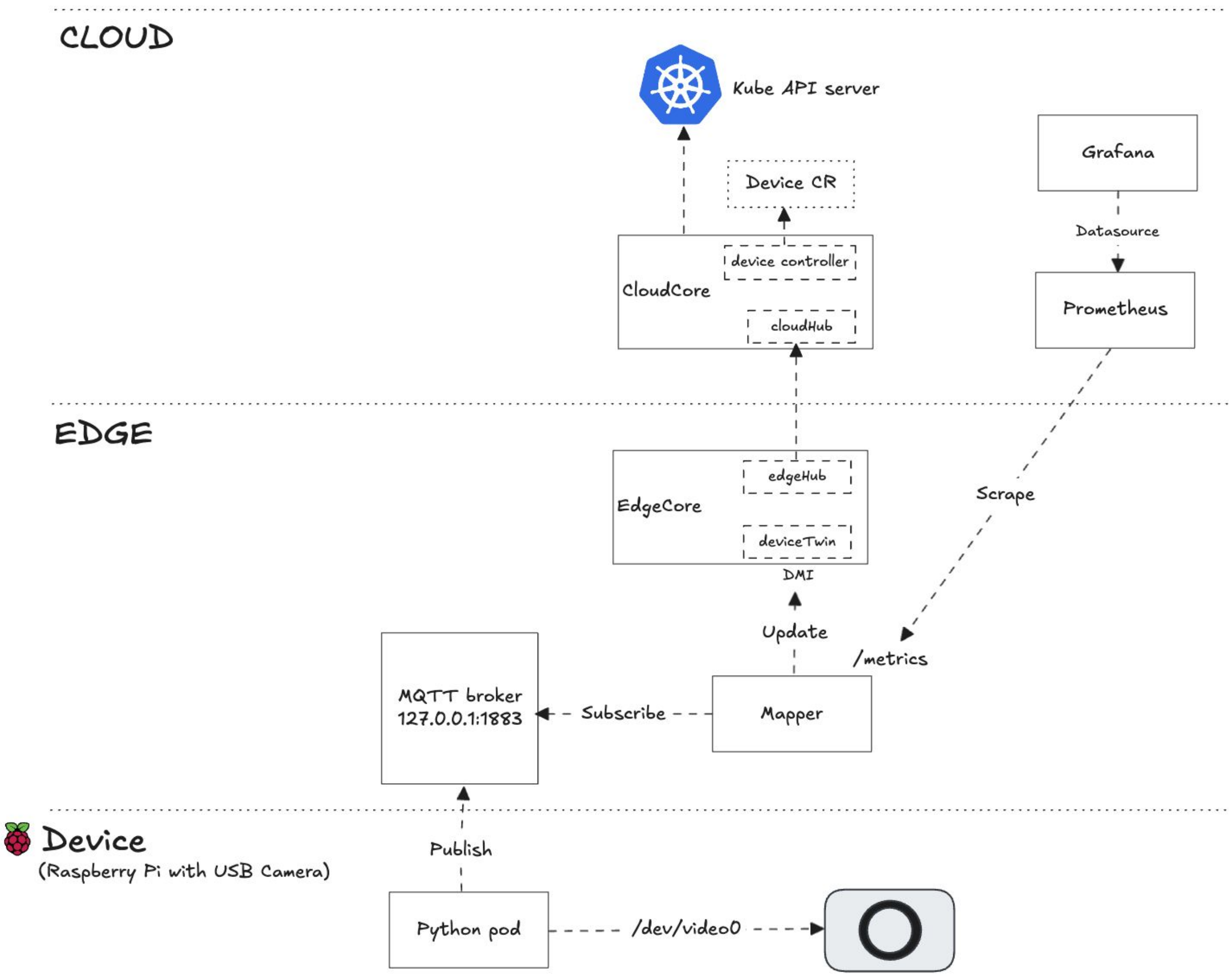
Switch to split terminal - I suggest:

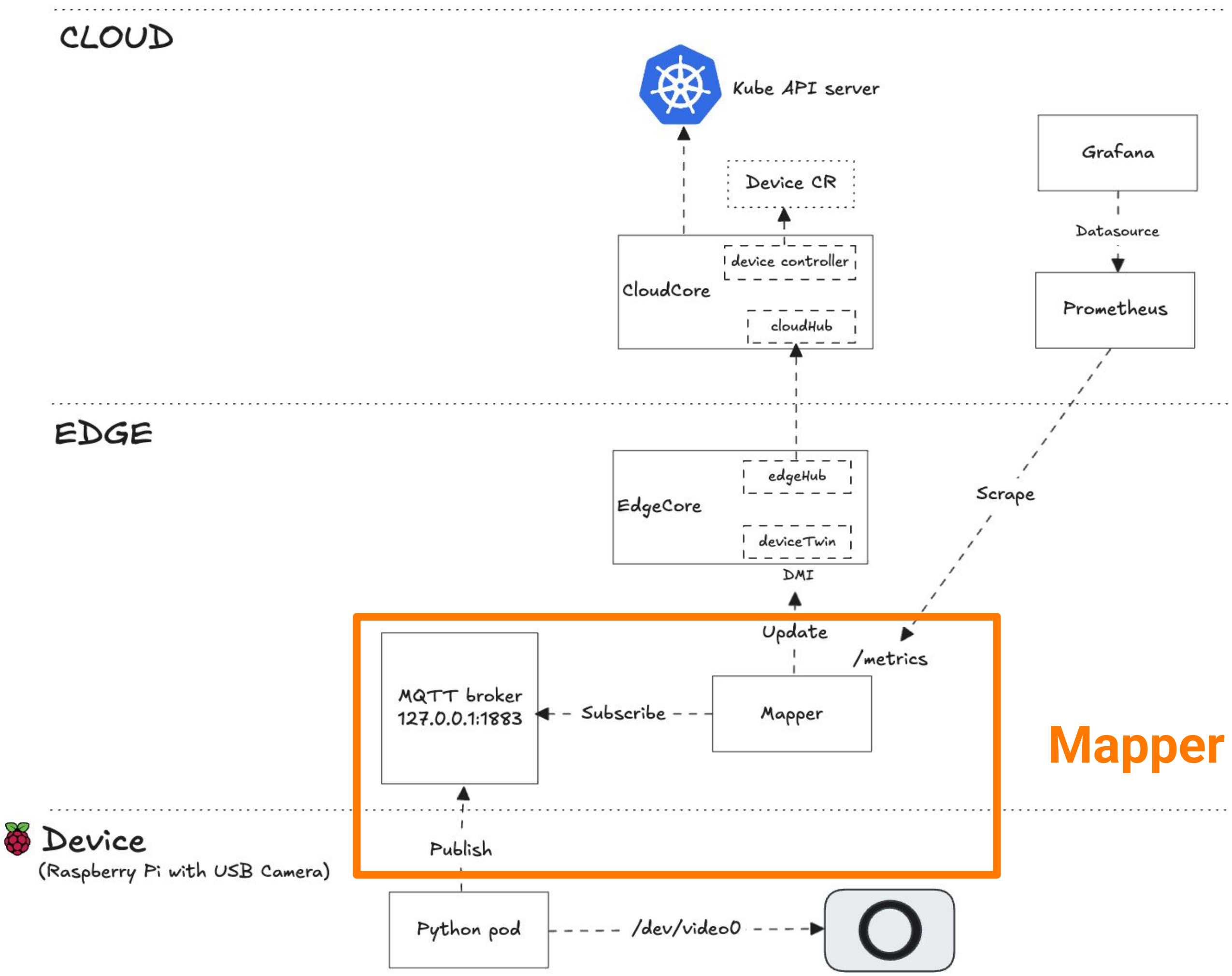
- `stern -n kubeedge -l app=mqtt-mapper -e 'too many request'`
- `ssh kubeedge@10.0.0.199 “journalctl -u edgecore -f”`
- `watch "echo Device instance CR hands value: \$(k get device hands -n default -ojsonpath='{.status.twins[0].reported.value}')"`
- `mosquitto_sub -h 10.0.0.199 -t '#' -v`

On Xav’s laptop, run `itermocil kubeedge-demo` to set these up

Maybe a few kubectl get commands (device, devicemodel)

SETUP


CLOUD
Kube API server
Device CR
Grafana
Datasource
device controller
CloudCore
cloudHub
Prometheus
EDGE
edgeHub
EdgeCore
deviceTwin
DMI
Scrape
Update
/metrics
MQTT broker
127.0.0.1:1883
Subscribe
Mapper
Device
(Raspberry Pi with USB Camera)
Publish
Python pod
/dev/video0


CLOUD
Kube API server
Device CR
Grafana
CloudCore
device controller
cloudHub
Datasource
Prometheus
EDGE
EdgeCore
edgeHub
deviceTwin
Scrape
DMI
Update
/metrics
MQTT broker
127.0.0.1:1883
Subscribe
Mapper
Mapper
Device
(Raspberry Pi with USB Camera)
Publish
Python pod
/dev/video0

Project repo `giantswarm/kubeedge-demo`

REALITY CHECK

Reality Check

Camera	»	*Industrial Sensor*
Raspberry Pi + Python app	»	*Edge Node + Analytics Model*
MQTT Mapper	»	*Protocol Adapters*
Device Twin, Device CR	»	*Digital Twin*
Prometheus and Grafana Dashboard	»	*Industrial Monitoring solutions*

THANK YOU